

# Tail-Aware Scheduling for Mixed LLM Workloads via Runtime Token-Based Demotion

Pavithra Selvaraj, Samiksha Ramakant Varpe, Shuran Sun

## 1 Motivation

Modern LLM serving systems like vLLM maximize throughput by continuously batching requests, but they treat all requests as equally expensive. In practice, workloads are highly heterogeneous: a one-sentence translation expects a near-instant response, while multi-step code generation may naturally take seconds. Under default First-Come-First-Serve (FCFS) scheduling, short requests stall behind long ones, driving up tail latency for the majority of users.

Prior work addresses this through learned length predictors that estimate output length before execution [1, 2, 3]. However, prediction at arrival time is fundamentally limited: the prompt alone carries weak signal about output length. Learning-to-Rank [1] achieves Kendall’s  $\tau \approx 0.62$  in-distribution but drops to 0.45 out-of-distribution, and TRAIL [2] reports that BERT-based prediction provides minimal improvement over FCFS. All learned predictors also require per-model retraining, limiting portability.

We take a different stance: rather than building better predictors, we rely on signals that emerge *during* generation. As the model decodes, two free signals become available - the number of tokens generated and the probability assigned to the EOS token at each step. These carry direct information about remaining length, grow more accurate over time, and require no training.

## 2 Research Question

**Can a scheduler relying solely on runtime signals - generated token count and the model’s per-step  $P(\text{EOS})$ , match or approach the tail-latency benefits of prediction-based schedulers, while eliminating the need for any learned predictor or per-model training?**

## 3 System Design

We propose a **two-queue MLFQ scheduler with runtime-only demotion**, built as a thin policy layer on top of vLLM. Continuous batching, KV cache management, and forward execution are unchanged

**Two-queue design.** We use two priority levels rather than a traditional  $N$ -level MLFQ. Since queue assignment relies solely on runtime signals, the decision boundary is naturally binary: *still observing* / *confirmed short* vs. *confirmed long*. All newly admitted requests enter the **probation queue** (high priority) by default - we make no attempt to classify at arrival time. Requests demoted from probation move to the **long queue** (low priority) with their KV cache preserved. A configurable quota (e.g., 20% of batch slots) is reserved for the long queue to prevent starvation.

**Runtime monitor (core contribution).** At every decode step, for each request in probation, the monitor records: (1) *token count*-free and model-agnostic; (2)  *$P(\text{EOS})$  trend* - the probability the model assigns to EOS, obtained from vLLM’s `logprobs` output. If EOS falls outside the returned top- $k$ , it is treated as zero. We track per-request  $\max(P(\text{EOS}))$  over the last  $N$  steps for noise robustness. The key insight is that  $P(\text{EOS})$  is a direct, unbiased signal of how close the model believes it is to finishing - unlike arrival-time predictors, its informativeness increases monotonically as generation proceeds.

**Decision rule** (applied once at step  $N$ ):

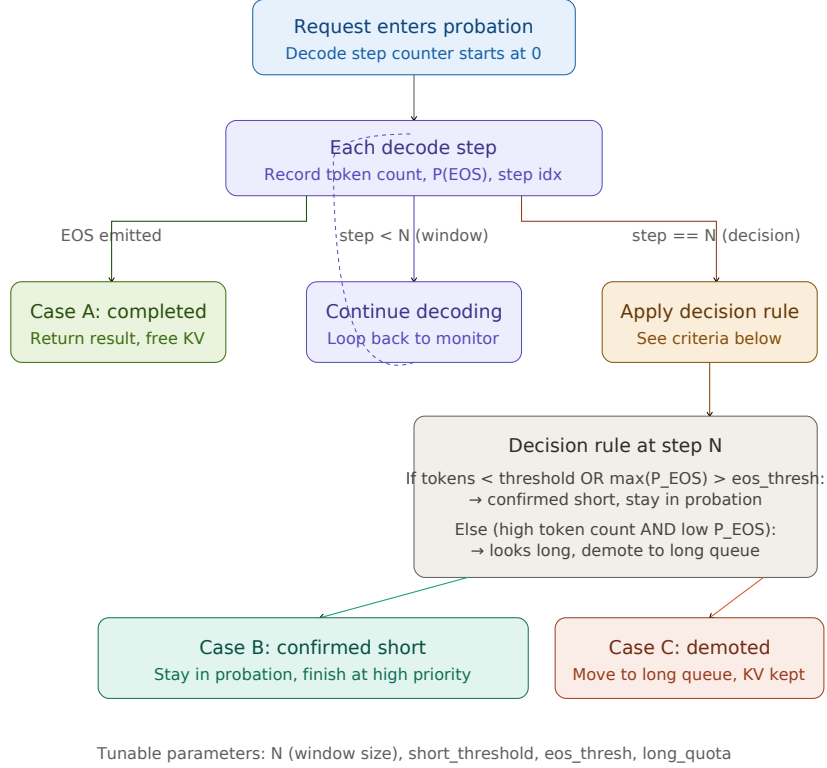


Figure 1: Per-request lifecycle: probation window, decision rule, and demotion.

```

if tokens < short_threshold OR max(P_EOS_history) > eos_thresh:
    -> confirmed short, stay in probation
else:
    -> confirmed long, demote to long queue
  
```

Demotion is **one-way** - no promotion back, avoiding oscillation (Figure 1). Tunable parameters:  $N$  (window size, 8–16), `short_threshold` (e.g., 64 tokens), `eos_thresh` (e.g., 0.05), `long_quota` (e.g., 0.20).

**Edge cases.** (1) *Starvation*: mitigated by the long-queue quota. (2) *Signal conflict* (low tokens, low  $P(\text{EOS})$  at step  $N$ ): we adopt an optimistic policy - keep in probation; if the request is truly long, token count will exceed the threshold soon. (3)  *$P(\text{EOS})$  noise*: max over the window provides natural smoothing; EWMA is a fallback if needed.

**Scoped-out extensions.** Arrival-time predictor (BERT/LtR), embedding-based corrector (TRAIL-style MLP), preemption with KV swap-out, and early rejection under overload are all left as optional future work. The minimal design is fully functional without them.

## 4 Evaluation Plan

We implement the scheduler as a modification to vLLM’s `v1/core/sched/scheduler.py` and evaluate on a single-node, single-GPU setup.

**Model.** Qwen2.5-7B-Instruct—manageable on one GPU, representative instruction-following behavior.

**Workloads.** Mixed traces combining short tasks (WMT translation, TriviaQA; <100 output tokens) with long tasks (GSM8K reasoning, HumanEval code generation; 200–2000 tokens). We vary the

mixture ratio (80/20, 50/50) and sweep arrival rate across low/medium/high load.

**Baselines.** (1) vLLM FCFS (default); (2) static heuristic (keyword-based arrival-time classification, no runtime demotion); (3) token-count-only ablation (our system with P(EOS) disabled). Baseline 2 isolates the contribution of runtime demotion; baseline 3 isolates the contribution of P(EOS).

**Metrics.** P90/P50 latency ratio (primary); P50/P90/P99 absolute latency by task type; overall throughput (tokens/s); demotion accuracy (fraction of demoted requests that are truly long); probation window overhead (added latency on short requests from the  $N$ -step window); per-request queue wait time by task type.

**Preliminary experiment.** Before integration, we will run an offline P(EOS) analysis: batch-process mixed requests with `logprobs` enabled, record per-step P(EOS) trajectories, and plot short-vs-long separation to validate signal quality and calibrate  $N$  and `eos_thresh`.

## 5 Related Work

**Length-aware scheduling.** vLLM-LtR [1] trains OPT-125M with ranking loss ( $\tau \approx 0.62$ , drops to 0.45 OOD). S3 [4] uses DistilBERT for coarse length buckets. PARS [3] extends this with pairwise ranking. All require per-model training and lack runtime feedback.

**Runtime-aware scheduling.** FastServe [5] applies skip-join MLFQ with fixed token budgets - the closest structural predecessor. However, it uses no P(EOS) signal and relies on proactive skip-join rather than observation-based demotion. TRAIL [2] trains an MLP on LLM hidden-state embeddings for runtime length refinement ( $2.66\times$  lower MAE than BERT), but is model-specific and requires retraining per LLM. Our approach uses P(EOS), which is model-agnostic and training-free.

**Overload protection.** Mooncake [6] introduces early rejection in a disaggregated KV-cache architecture. SCORPIO and Niyama explore proactive rejection and service degradation. These are orthogonal and listed as optional extensions in our design.

## 6 Resources

A single GPU (A100 or equivalent), the open-source vLLM framework, and Qwen2.5-7B-Instruct. Workloads from public datasets (WMT, TriviaQA, GSM8K, HumanEval). No proprietary data, closed models, or multi-node infrastructure needed. The scope is intentionally limited to single-node with no changes to vLLM’s attention kernels or distributed infrastructure.

## References

- [1] Mao et al., “Efficient LLM Scheduling by Learning to Rank,” ICLR 2025. [arXiv:2408.15792](#)
- [2] Qiu et al., “Efficient Interactive LLM Serving with Proxy Model-based Sequence Length Prediction,” ICLR 2025. [arXiv:2410.01035](#)
- [3] Chen et al., “PARS: Prompt-Aware pairwise Ranking-based Scheduling,” 2025. [arXiv:2510.03243](#)
- [4] Jin et al., “S3: Increasing GPU Utilization during Generative Inference for Higher Throughput,” NeurIPS 2024. [arXiv:2401.08671](#)
- [5] Wu et al., “Fast Distributed Inference Serving for Large Language Models,” 2024. [arXiv:2305.05920](#)
- [6] Qin et al., “Mooncake: A KVCache-centric Disaggregated Architecture for LLM Serving,” FAST 2025. [arXiv:2407.00079](#)